

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 311 — Past Questions & Materials
by Renance

Course: CSC 311
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 4
Levels: 300

This study pack was compiled and restructured by Renance for CSC 311 students at Federal University of Petroleum Resources, Effurun. It brings together 4 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 4 — CSC 311 Structured Programming 2024/2025

Level 300 • First Semester • PastQuestion

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

DEPARTMENT OF COMPU? BR SCIENCE

18) SEMESTER UNDERGRADUATE EXAMINATION (2024/2025 SESSION)

CSCI: STRUCTURED PROGRAMMING

INSTRUCTIONS: Date (Time): 7 May 2025 (11am)

1. Attempt Any Four (4) Questions, One (1) Compulsory.

>. 'Vine Allowed: 2 Hours.

A. Answer tach Question On A New Page.

rae PS

Question 1

a) What are the following terms: 16 marks}

i, Control Structures (~~ x

ii. Flowcharting <—~ ,

iii, Branching aK 6

b) Briefly explain the three (3) main/categories of Control Structures. a [6 marks]

c) What is Structured programming? List five (5) elements of structured programming. {7 marks]

Question 2

a) What is the meaning of the terms (i) Spaghetti code (ii) Structured design principles? [4 marks]

b) Enumerate the four (4) basic structured design principles? [8 marks}

c) Give Se benefits of structured design, in structured programming. 5 [5 marks]

Qeestion 3

a) Explain briefly thé terms by giving three (3) highlights wach foc i) Abstraction (if) Modularity [6 marks]

b) List four (4) benefits of abstraction and modularity. yD . !4 marks]

c) Whatis Stepwise refinement? Outline the six (6) steps of Stzpwise refinement. , ~ [7 marks]

Question 4 ‘

a) Briefly explain the term “structured design techniques”? Lisi any five (5). {7 marks]

b) Give simple meanings for these methodologies of structured programming: [5 marks]

i. Hands-on practice

ii. Problem-solving approach

iii. Debugging techniques

iv. Visual aids

v. Real-world examples

c) Explain the tern: “Pseudocode”. List three (3) examples c! structured programming languages. {5 marks]

15 marks]

~ Question 5 VA

- a) Outline five characteristics of a good program. > '
- b) What are the eight (8) phases of program development (programming)? [8 marks]
- c) Give four (4) advantages of structured programming. ad [4 marks]

Paper 2 of 4 — CSC 311 Structured Programming: (3 Units)

Level 300 • First Semester • Handout

CSC 311 Structured Programming: (3 Units)

Structured Programming elements, structured design principles, abstraction, modularity, stepwise refinement, structured design techniques. Teaching of a structured programming language etc.

Lectures

A lead to “spaghetti code” that is potentially difficult to follow and maintain.[1]

Discussion

One of the most important concepts of programming is the ability to control a program so that different lines of code are executed or that some lines of code are executed many times. The mechanisms that allow us to control the flow of execution are called control structures. Flowcharting is a method of documenting (charting) the flow (or paths) that a program would execute. There are three main categories of control structures:

- Sequence – Very boring. Simply do one instruction then the next and the next. Just do them in a given sequence or in the order listed. Most lines of code are this.
- Selection – This is where you select or choose between two or more flows. The choice is decided by asking some sort of question. The answer determines the path (or which lines of code) will be executed.
- Iteration – Also known as repetition, it allows some code (one to many lines) to be executed (or repeated) several times. The code might not be executed at all (repeat it zero times), executed a fixed number of times or executed indefinitely until some condition has been met. Also known as looping because the flowcharting shows the flow looping back to repeat the task.

A fourth category describes unstructured code.

- Branching – An uncontrolled structure that allows the flow of execution to jump to a different part of the program. This category is rarely used in modular structured programming.

All high-level programming languages have control structures. All languages have the first three categories of control structures (sequence, selection, and iteration). Most have the if then else structure (which belongs to the selection category) and the while structure (which belongs to the iteration category). After these two basic structures, there are usually language variations.

The concept of structured programming started in the late 1960's with an article by Edsger Dijkstra. He proposed a “go to less” method of planning programming logic that eliminated the need for the branching category of control structures. The topic was debated for about 20 years. But ultimately – “By the end of the 20th century

nearly all computer scientists were convinced that it is useful to learn and apply the concepts of structured programming.”[2]

Key Terms

branching

An uncontrolled structure that allows the flow of execution to jump to a different part of the program.

control structures

Mechanisms that allow us to control the flow of execution within a program.

iteration

A control structure that allows some lines of code to be executed many times.

selection

A control structure where the program chooses between two or more options.

sequence

A control structure where the program executes the items in the order listed.

spaghetti code

A pejorative phrase for unstructured and difficult to maintain source code.[3]

structured programming

A method of planning programs that avoids the branching category of control structures.

References

- cnx.org: Programming Fundamentals – A Modular Structured Approach using C++

Elements of structured programming include:

- Sequence: The basic structure of a programming language, where code statements are executed in order
- Selection: The execution of code blocks based on the result of a condition
- Control structures: Mechanisms that control the flow of a program
- Procedures: Named sets of instructions that are analogous to functions
- Conditional statements: Keywords like "if", "then", "else", and "endif" that are used in selection

Structured programming is a programming paradigm that emphasizes clarity and simplicity. It's the basis for good programming style.

Explanation

- Sequence: The simplest form of control flow, where statements are executed in the order they appear in the code
- Selection: Uses conditional statements to execute code blocks based on the result of a condition
- Control structures: Mechanisms that control the flow of a program, such as sequence
- Procedures: Named sets of instructions that are similar to functions

Structured design principles are a set of guidelines for building systems from smaller,

self-contained components. These principles ensure that the final system is well-documented and can be tested as a whole.

Principles

- Group related parts: Put parts of a problem that are related into the same part of the system.
- Separate unrelated parts: Put parts of a problem that are unrelated into separate parts of the system.
- Use self-documenting names: Name each subcomponent of the system to reflect its purpose.
- Test subcomponents separately: Test each subcomponent as if it were a separate system.

Benefits

- Structured design helps ensure that the final system is well-documented.
- Structured design helps ensure that the final system can be tested as a whole

Abstraction and modularity are software design principles that help developers create and maintain complex systems. Abstraction is the process of hiding details and focusing on the essential functionality of something. Modularity is the process of breaking a system down into smaller, independent, and reusable components.

Abstraction

- Focuses on the important parts of something, while hiding the complicated details
- Helps users use something without needing to understand all of its inner workings
- In computer science, abstraction can be described at different levels, such as physical, logical, and view

Modularity

- Breaks a system down into smaller, more manageable parts
- Makes it easier to change parts without affecting other parts
- Cohesive and loosely coupled components are easier to understand, reuse, and replace

Benefits of abstraction and modularity

- Improves the readability, maintainability, testability, and scalability of software

Stepwise refinement is a programming technique that breaks down a problem into smaller, more manageable components. It's also known as top-down design.

Steps of stepwise refinement

1. Start with the problem statement
2. Break the problem into general steps
3. Break each step into more detailed steps
4. Repeat the process until the problem is broken down into specific steps
5. Develop and refine each component
6. Compile, execute, and test each step
- 7.

Why use stepwise refinement?

- Stepwise refinement helps programmers gain insight into the compositional

subsystems of a system

- It allows programmers to introduce programming constructs progressively
- It helps programmers produce code that correctly implements a specification

How is stepwise refinement implemented?

- Stepwise refinement is based on pseudo-code, a low-level design tool
- Each module can be tested separately or form part of a library routine

Structured design techniques are methods used in software engineering to organize and structure the design of a computer program, particularly within the context of structured programming. Structured programming is a programming paradigm that emphasizes the use of structured control flow constructs, such as loops and conditionals, to improve the clarity and quality of software.□

Here are some key structured design techniques:

1. **Top-Down Design:**

- In top-down design, you start with a high-level view of the system and then systematically break it down into smaller and more manageable sub-systems or modules.
- Each module is further decomposed until you reach a level where the implementation details can be easily handled.

2. **Modularity:**

- Modularity is a fundamental principle in structured design. It involves breaking down a complex system into smaller, self-contained modules or functions.
- Each module should perform a specific, well-defined task, and these modules should interact with each other through well-defined interfaces.

3. **Stepwise Refinement:**

- This technique involves refining each module or function in a stepwise manner until the implementation details are clear.
- The process involves starting with an abstract or high-level description and refining it through a series of steps until you have a detailed design.

4. **Data Flow Diagrams (DFD):**

- DFDs are graphical tools used for representing the flow of data within a system. They help visualize how data moves through different processes in a system.
- DFDs are particularly useful for understanding the relationships between different components and for identifying potential areas of improvement.

5. **Hierarchy:**

- Structured design often involves organizing the program into a hierarchical structure. This hierarchy reflects the organization of the modules and their relationships.
- The top-level module may represent the main program, and lower-level modules may represent subroutines or functions.

6. **Control Flow Diagrams (CFD):**

- CFDs illustrate the control flow of a program. They show the sequence of execution of different parts of a program.
- By using constructs such as loops and conditionals, CFDs help in visualizing the flow of control through the program.

7. **Pseudocode:**

- Pseudocode is an informal way of expressing the design of a computer program. It uses high-level statements that are similar to a programming language but are not meant to be executed.
- Pseudocode helps in clarifying the logic of the program before the actual coding begins.

Structured design techniques aim to improve the clarity, maintainability, and modularity of software systems. They are particularly useful in large-scale software development projects where a systematic and organized approach is essential for managing complexity.

A structured programming language involves focusing on the core concepts of sequence, selection (if/else statements), and iteration (loops) to build programs with clear logic, breaking down complex problems into smaller, manageable modules, and avoiding unstructured jumps like "goto" statements, all while emphasizing proper syntax and best practices within the chosen language; commonly used languages for teaching structured programming include Pascal and C.

Key aspects of structured programming:

- Fundamental concepts:
- Variables and data types: Understanding different data types (integers, floats, strings, booleans) and how to declare and manipulate variables.
- Operators: Arithmetic, logical, comparison operators and their usage in expressions.
- Input and output: How to take user input and display results using functions like print or scanf.
- Control flow structures:
- Sequence: Executing code in a linear order, one statement after another.
- Selection (if-else): Making decisions based on conditions, choosing which block of code to execute.
- Iteration (loops): Repeating a block of code multiple times based on a condition (while, for loops).
- Functions:
- Concept of functions: Defining reusable blocks of code that perform specific tasks.
- Function parameters and return values: Passing data into functions and receiving results.
- Function calls: Executing a function by its name.
- Structured programming principles:

- Top-down design: Breaking a problem into smaller, more manageable sub-problems.
 - Modularity: Dividing code into well-defined functions with specific responsibilities.
 - Readability and maintainability: Writing code that is easy to understand and modify.
 - Avoiding "goto" statements: Relying solely on structured control flow mechanisms.
- methodology:
- Hands-on practice: write code from the beginning, starting with simple exercises and gradually increasing complexity.
 - Problem-solving approach: go through the process of analysing problems, designing algorithms, and translating them into code.
 - Debugging techniques: Identify and fix errors in the code.
 - Visual aids: Use flowcharts to illustrate program logic and control flow.
 - Real-world examples: Relate programming concepts to practical applications relating to the' interests.

Example structured programming languages for teaching:

- Pascal:

Considered a classic language specifically designed to promote structured programming concepts.

- C:

A widely used language with a good balance between simplicity and power, enabling students to learn structured programming principles while building more complex applications.

- Python:

While considered more versatile, Python also allows for teaching structured programming concepts due to its clear syntax and emphasis on readability.

PROGRAMMING LANGUAGES

INTRODUCTION

A program is a finite set of sequenced instructions or commands given to a computer in order to carry out a particular task. To write a program for the computer to carry out these instructions, there must be a means of communication. Humans communicate via natural languages such as English, French, Chinese etc. Likewise, to communicate with the computer, we also use languages known as programming languages.

Programming is the art of program writing using a particular programming language. It can also be said to be the process of writing a set of instructions in sequential manner using programming language to control the activity of a computer system. A computer programming language is an artificial language which is used in writing a set of formal instructions to control the activities of a computer system.

There are two main types of computer programming languages; these are: Low-level language and High-level language. Low-level languages also known as machine language, are machine dependent and makes fast and efficient use of the computer. It requires no translator to translate the code. It is directly understood by the computer. On the contrary, writing a program in high-level language does not require knowledge

of the computer in which the program is run. Hence, high-level languages are machine independent, and which programs are portable to other computers, and very easy to learn and write.

Low level language

This type of language is closer to the machine compared with the human or natural language. The two major examples are the Machine language and the Assembly language.

Machine Language: This is the only language computer understands. It is the native language of the computer. The computer directly executes a program written in machine language. These programs are coded using strings of 0's and 1's. It doesn't need a translator.

Advantages of Machine Language

Machine language makes fast and efficient use of the computer.

It requires no translator to translate the code. It is directly understood by the computer.

Disadvantages of Machine Language

Very bulky.

They require much time for writing and reading.

They are prone to error which is difficult to detect and correct.

Very difficult to learn.

Can only run on the computer it is designed (i.e., it is machine dependent)

Assembly Language: Assembly Language uses MNEMONICS (symbols) to represent data and instructions. Such program eliminates problems associated with machine language. Computer cannot execute directly a program written in assembly language; it requires a translator called assembler. Assembler is a special program designed to translate a program written in assembly language to a machine language equivalent.

Advantages of Assembly Language

It allows complex jobs to run in a simpler way.

It is memory efficient, as it requires less memory.

It is faster in speed, as its execution time is less.

It is mainly hardware oriented.

It requires less instruction to get the result.

It is used for critical jobs.

Disadvantages of Assembly Language

It is machine dependent; the programmer must be knowledgeable in both subject area and the operations of the machine.

It is cumbersome though less cumbersome than that of machine language.

Very expensive to develop

It consumes time

High Level Language

A high-level language is a problem-orientated programming language. The source programs are written in human readable languages like English instead of mere symbols. In other words, a high-level language is a convenient and simple means of describing the information structures and sequences of actions required to perform a particular task.

Advantages of High-Level Language

No knowledge of the computer in which the program will be run is required

The programs are portable

Very easy to learn and write

Disadvantages of High-Level Language

It takes additional translation times to translate the source to machine code.

High level programs are comparatively slower than low level programs.

Compared to low level programs, they are generally less memory efficient.

Cannot communicate directly with the hardware

Features of High-Level Language

Machine independent

Problem oriented

Ability to clearly reflect the structure of program written in it.

Readability

Programs are portable.

Examples of High-level Languages are FORTRAN, COBOL, QBASIC, VISUAL BASIC, JAVA, PASCAL etc.

Characteristics of a Good Program

Portability/Transferability: Must be able to work on any computer machine.

Reliability: It can be relied upon to do what it is expected to do.

Efficiency/cost saving: It must not cost more than its benefits and enables problem to be solved appropriately, quickly and efficiently.

Clarity and Simplicity: It should be as simple as possible to understand.

Understandability/Readability: It must be readable and understandable by other programmers and end users.

Flexibility/Adaptability/Maintainability: A good program must be flexible adaptable and maintainable in order to suit user's need.

Modification must be possible and very easy.

Phases of Program Development (Programming)

The process of producing a computer program (software) may be divided into eight phases or stages:

- 1) Problem definition/Analysis
- 2) Selection or development of an algorithm
- 3) Designing the program
- 4) Coding the programming statements

- 5) Compiling/Compilation stage
- 6) Testing/Running and Debugging the program
- 7) Documentation.
- 8) Maintenance

1) Problem Definition/Analysis Stage: There is need to understand the problem that requires a solution. The need to determine the data to be processed, form or type of the data, volume of the data, what to be done to the data to produce the expected/required output.

2) Selection or development of an algorithm: An algorithm is the set of steps required to solve a problem written down in English language.

3) Designing the program: In order to minimize the amount of time to be spent in developing the software, the programmer makes use of flowchart. Flowchart is the pictorial representation of the algorithm developed in step 2 above. Pseudocode IPO chart (input processing output) and HIPO (Hierarchical Input Processing and Output) chart may be used in place of flowchart or to supplement flowchart.

4) Coding the statement: This involves writing the program statements. The programmer uses the program flow chart as a guide for coding the steps the computer will follow.

5) Compiling: There is need to translate the program from the source code to the machine or object code if it is not written in machine language. A computer program is fed into the computer first, then as the source program is entered, a translated equivalent (object program) is created and stored in the memory.

6) Running, Testing and Debugging: When the computer is activated to run a program, it may find it difficult to run it because errors (syntax, semantics or logic, or runtime) might have been committed. Manuals are used to debug the errors. A program that is error free is tested using some test data. If the program works as intended, real life data are then loaded.

7) Documentation: This is the last stage in software development. This involves keeping written records that describe the program, explain its purposes, define the amount, types and sources of input data required to run it. List the Departments and people who use its output and trace the logic the program follows.

8) Maintenance: All the activities that occur after the completion of the program come under the program maintenance. Program maintenance includes the following: Finding and correcting the errors; Modifying the program to enhance it – i.e., adapting to some new concepts or when there is a change in the hardware or operating system; Update the documentation; Add new features and functions; Remove useless or redundant parts of code.

Structured programming Concept

In structured programming design, programs are broken into different functions these functions are also known as modules, subprogram, subroutines or procedures.

Each function is design to do a specific task with its own data and logic. Information

can be passed from one function to another function through parameters. A function can have local data that cannot be accessed outside the function's scope. The result of this process is that all the other different functions are synthesized in another function. This function is known as main function. Many of the high-level languages support structured programming.

Structured programming minimizes the chances of the function affecting another. It allows for clearer programs code. It made global variables to disappear and replaced by the local variables. Due to this change one can save the memory allocation space occupied by the global variable.

Its organization helps in the easy understanding of programming logic. So that one can easily understand the logic behind the programs. It also

Helps the newcomers of any industrial technology company to understand the programs created by their senior workers of the industry.

The languages that support Structured programming approach are:

- C
- C++
- Java
- C#
- Pascal

Python

Fortran

Cobol

Advantages of Structured programming

It is user friendly and easy to understand.

Similar to English vocabulary of words and symbols.

It is easier to learn.

They require less time to write.

They are easier to maintain.

These are mainly problem oriented rather than machine based.

Program written in a higher-level language can be translated into many machine languages and therefore can run on any computer for which there exists an appropriate translator.

It is independent of machine on which it is used, i.e., programs developed in high level languages can be run on any computer.

Disadvantages of Structured Programming

Structured programming codes implemented in high-level language has to be translated into the machine language by translator and thus a price in computer time is paid.

The object code generated by a translator might be inefficient compared to an equivalent assembly language program.

Data type are proceeds in many functions in a structured program. When changes

occur in those data types, the corresponding change must be made to every location that acts on those data types within the program. This is really a very time consuming task if the program is very large.

In a structured program, each programmer is assigned to build a specific set of functions and data types. Since different programmers handle separate functions that have mutually shared data type, other programmers in the team must reflect the changes in data types done by the programmer in data type handled. Otherwise, it requires rewriting several functions.

Programming paradigms

Solving a programming problem requires choosing the right concepts. All but the smallest toy problems require different sets of concepts for different parts of the program. A programming paradigm, or programming model, is an approach to programming a computer based on a mathematical theory or a coherent set of principles. It is a way of conceptualizing what it means to perform computation and how tasks to be carried out on the computer should be structured and organized.

Programming languages are used to realize programming paradigms.

Examples of programming paradigms are: imperative, functional, logical, object-oriented. Most popular languages are imperative and use structured programming techniques. Structured programming techniques involve giving the code you write structures, these often involve writing code in blocks such as sequence (code executed line by line), selection (branching statements such as if..then..else, or case) and repetition (iterative statements such as for, while, repeat, loop).

Imperative paradigm

This paradigm is based on the ideas of a Von Neumann architecture. A command has a measurable effect on the program and the order of commands is important. First do this and next do that. Its main characteristics are incremental change of the program state (variables) as a function of time; execution of commands in an order governed by

control structures; and the use of procedures, abstractions of one or more actions, which can be called as a single command. Examples: Fortran, Algol, Basic, C, Pascal.

Functional paradigm

This paradigm is based on mathematics and theory of functions. The values produced are non-mutable and plays a minor role compared to imperative program. All computations are done by applying functions with no side effects. Functions are first class citizens. Evaluate an expression and use the resulting value for something.

Example: Haskell.

Logical paradigm

The logic paradigm fits well when applied in problem domains that deal with the extraction of knowledge from basic facts and relations. Is based on axioms, inference rules, and queries. Program execution becomes a systematic search in a set of facts, making use of a set of inference rules. Examples: Prolog and List.

Object-oriented paradigm

Data as well as operations are encapsulated in objects. Information hiding is used to protect internal properties of an object. Objects interact by means of message passing. In most object-oriented languages, objects are grouped in classes and classes are organized as hierarchies. Examples: C++, Java.

Other Paradigms

Other paradigms include Visual paradigm, Constraint based paradigm, Aspect oriented paradigm and Event-oriented paradigm.

Paper 3 of 4 — CSC 311 Structured Programming: (3 Units)

Level 300 • First Semester • Handout

CSC 311 Structured Programming: (3 Units)

Structured Programming elements, structured design principles, abstraction, modularity, stepwise refinement, structured design techniques. Teaching of a structured programming language etc.

Lectures

A lead to “spaghetti code” that is potentially difficult to follow and maintain.[1]

Discussion

One of the most important concepts of programming is the ability to control a program so that different lines of code are executed or that some lines of code are executed many times. The mechanisms that allow us to control the flow of execution are called control structures. Flowcharting is a method of documenting (charting) the flow (or paths) that a program would execute. There are three main categories of control structures:

- Sequence – Very boring. Simply do one instruction then the next and the next. Just do them in a given sequence or in the order listed. Most lines of code are this.
- Selection – This is where you select or choose between two or more flows. The choice is decided by asking some sort of question. The answer determines the path (or which lines of code) will be executed.
- Iteration – Also known as repetition, it allows some code (one to many lines) to be executed (or repeated) several times. The code might not be executed at all (repeat it zero times), executed a fixed number of times or executed indefinitely until some condition has been met. Also known as looping because the flowcharting shows the flow looping back to repeat the task.

A fourth category describes unstructured code.

- Branching – An uncontrolled structure that allows the flow of execution to jump to a different part of the program. This category is rarely used in modular structured programming.

All high-level programming languages have control structures. All languages have the first three categories of control structures (sequence, selection, and iteration). Most have the if then else structure (which belongs to the selection category) and the while structure (which belongs to the iteration category). After these two basic structures, there are usually language variations.

The concept of structured programming started in the late 1960's with an article by Edsger Dijkstra. He proposed a “go to less” method of planning programming logic that eliminated the need for the branching category of control structures. The topic was debated for about 20 years. But ultimately – “By the end of the 20th century

nearly all computer scientists were convinced that it is useful to learn and apply the concepts of structured programming.”[2]

Key Terms

branching

An uncontrolled structure that allows the flow of execution to jump to a different part of the program.

control structures

Mechanisms that allow us to control the flow of execution within a program.

iteration

A control structure that allows some lines of code to be executed many times.

selection

A control structure where the program chooses between two or more options.

sequence

A control structure where the program executes the items in the order listed.

spaghetti code

A pejorative phrase for unstructured and difficult to maintain source code.[3]

structured programming

A method of planning programs that avoids the branching category of control structures.

References

- cnx.org: Programming Fundamentals – A Modular Structured Approach using C++

Elements of structured programming include:

- Sequence: The basic structure of a programming language, where code statements are executed in order
- Selection: The execution of code blocks based on the result of a condition
- Control structures: Mechanisms that control the flow of a program
- Procedures: Named sets of instructions that are analogous to functions
- Conditional statements: Keywords like "if", "then", "else", and "endif" that are used in selection

Structured programming is a programming paradigm that emphasizes clarity and simplicity. It's the basis for good programming style.

Explanation

- Sequence: The simplest form of control flow, where statements are executed in the order they appear in the code
- Selection: Uses conditional statements to execute code blocks based on the result of a condition
- Control structures: Mechanisms that control the flow of a program, such as sequence
- Procedures: Named sets of instructions that are similar to functions

Structured design principles are a set of guidelines for building systems from smaller,

self-contained components. These principles ensure that the final system is well-documented and can be tested as a whole.

Principles

- Group related parts: Put parts of a problem that are related into the same part of the system.
- Separate unrelated parts: Put parts of a problem that are unrelated into separate parts of the system.
- Use self-documenting names: Name each subcomponent of the system to reflect its purpose.
- Test subcomponents separately: Test each subcomponent as if it were a separate system.

Benefits

- Structured design helps ensure that the final system is well-documented.
- Structured design helps ensure that the final system can be tested as a whole

Abstraction and modularity are software design principles that help developers create and maintain complex systems. Abstraction is the process of hiding details and focusing on the essential functionality of something. Modularity is the process of breaking a system down into smaller, independent, and reusable components.

Abstraction

- Focuses on the important parts of something, while hiding the complicated details
- Helps users use something without needing to understand all of its inner workings
- In computer science, abstraction can be described at different levels, such as physical, logical, and view

Modularity

- Breaks a system down into smaller, more manageable parts
- Makes it easier to change parts without affecting other parts
- Cohesive and loosely coupled components are easier to understand, reuse, and replace

Benefits of abstraction and modularity

- Improves the readability, maintainability, testability, and scalability of software

Stepwise refinement is a programming technique that breaks down a problem into smaller, more manageable components. It's also known as top-down design.

Steps of stepwise refinement

1. Start with the problem statement
2. Break the problem into general steps
3. Break each step into more detailed steps
4. Repeat the process until the problem is broken down into specific steps
5. Develop and refine each component
6. Compile, execute, and test each step
- 7.

Why use stepwise refinement?

- Stepwise refinement helps programmers gain insight into the compositional

subsystems of a system

- It allows programmers to introduce programming constructs progressively
- It helps programmers produce code that correctly implements a specification

How is stepwise refinement implemented?

- Stepwise refinement is based on pseudo-code, a low-level design tool
- Each module can be tested separately or form part of a library routine

Structured design techniques are methods used in software engineering to organize and structure the design of a computer program, particularly within the context of structured programming. Structured programming is a programming paradigm that emphasizes the use of structured control flow constructs, such as loops and conditionals, to improve the clarity and quality of software.□

Here are some key structured design techniques:

1. **Top-Down Design:**

- In top-down design, you start with a high-level view of the system and then systematically break it down into smaller and more manageable sub-systems or modules.
- Each module is further decomposed until you reach a level where the implementation details can be easily handled.

2. **Modularity:**

- Modularity is a fundamental principle in structured design. It involves breaking down a complex system into smaller, self-contained modules or functions.
- Each module should perform a specific, well-defined task, and these modules should interact with each other through well-defined interfaces.

3. **Stepwise Refinement:**

- This technique involves refining each module or function in a stepwise manner until the implementation details are clear.
- The process involves starting with an abstract or high-level description and refining it through a series of steps until you have a detailed design.

4. **Data Flow Diagrams (DFD):**

- DFDs are graphical tools used for representing the flow of data within a system. They help visualize how data moves through different processes in a system.
- DFDs are particularly useful for understanding the relationships between different components and for identifying potential areas of improvement.

5. **Hierarchy:**

- Structured design often involves organizing the program into a hierarchical structure. This hierarchy reflects the organization of the modules and their relationships.
- The top-level module may represent the main program, and lower-level modules may represent subroutines or functions.

6. **Control Flow Diagrams (CFD):**

- CFDs illustrate the control flow of a program. They show the sequence of execution of different parts of a program.
- By using constructs such as loops and conditionals, CFDs help in visualizing the flow of control through the program.

7. **Pseudocode:**

- Pseudocode is an informal way of expressing the design of a computer program. It uses high-level statements that are similar to a programming language but are not meant to be executed.
- Pseudocode helps in clarifying the logic of the program before the actual coding begins.

Structured design techniques aim to improve the clarity, maintainability, and modularity of software systems. They are particularly useful in large-scale software development projects where a systematic and organized approach is essential for managing complexity.

A structured programming language involves focusing on the core concepts of sequence, selection (if/else statements), and iteration (loops) to build programs with clear logic, breaking down complex problems into smaller, manageable modules, and avoiding unstructured jumps like "goto" statements, all while emphasizing proper syntax and best practices within the chosen language; commonly used languages for teaching structured programming include Pascal and C.

Key aspects of structured programming:

- Fundamental concepts:
- Variables and data types: Understanding different data types (integers, floats, strings, booleans) and how to declare and manipulate variables.
- Operators: Arithmetic, logical, comparison operators and their usage in expressions.
- Input and output: How to take user input and display results using functions like print or scanf.
- Control flow structures:
- Sequence: Executing code in a linear order, one statement after another.
- Selection (if-else): Making decisions based on conditions, choosing which block of code to execute.
- Iteration (loops): Repeating a block of code multiple times based on a condition (while, for loops).
- Functions:
- Concept of functions: Defining reusable blocks of code that perform specific tasks.
- Function parameters and return values: Passing data into functions and receiving results.
- Function calls: Executing a function by its name.
- Structured programming principles:

- Top-down design: Breaking a problem into smaller, more manageable sub-problems.
 - Modularity: Dividing code into well-defined functions with specific responsibilities.
 - Readability and maintainability: Writing code that is easy to understand and modify.
 - Avoiding "goto" statements: Relying solely on structured control flow mechanisms.
- methodology:
- Hands-on practice: write code from the beginning, starting with simple exercises and gradually increasing complexity.
 - Problem-solving approach: go through the process of analysing problems, designing algorithms, and translating them into code.
 - Debugging techniques: Identify and fix errors in the code.
 - Visual aids: Use flowcharts to illustrate program logic and control flow.
 - Real-world examples: Relate programming concepts to practical applications relating to the' interests.

Example structured programming languages for teaching:

- Pascal:

Considered a classic language specifically designed to promote structured programming concepts.

- C:

A widely used language with a good balance between simplicity and power, enabling students to learn structured programming principles while building more complex applications.

- Python:

While considered more versatile, Python also allows for teaching structured programming concepts due to its clear syntax and emphasis on readability.

PROGRAMMING LANGUAGES

INTRODUCTION

A program is a finite set of sequenced instructions or commands given to a computer in order to carry out a particular task. To write a program for the computer to carry out these instructions, there must be a means of communication. Humans communicate via natural languages such as English, French, Chinese etc. Likewise, to communicate with the computer, we also use languages known as programming languages. Programming is the art of program writing using a particular programming language. It can also be said to be the process of writing a set of instructions in sequential manner using programming language to control the activity of a computer system. A computer programming language is an artificial language which is used in writing a set of formal instructions to control the activities of a computer system.

There are two main types of computer programming languages; these are: Low-level language and High-level language. Low-level languages also known as machine language, are machine dependent and makes fast and efficient use of the computer. It requires no translator to translate the code. It is directly understood by the computer. On the contrary, writing a program in high-level language does not require knowledge

of the computer in which the program is run. Hence, high-level languages are machine independent, and which programs are portable to other computers, and very easy to learn and write.

Low level language

This type of language is closer to the machine compared with the human or natural language. The two major examples are the Machine language and the Assembly language.

Machine Language: This is the only language computer understands. It is the native language of the computer. The computer directly executes a program written in machine language. These programs are coded using strings of 0's and 1's. It doesn't need a translator.

Advantages of Machine Language

Machine language makes fast and efficient use of the computer.

It requires no translator to translate the code. It is directly understood by the computer.

Disadvantages of Machine Language

Very bulky.

They require much time for writing and reading.

They are prone to error which is difficult to detect and correct.

Very difficult to learn.

Can only run on the computer it is designed (i.e., it is machine dependent)

Assembly Language: Assembly Language uses MNEMONICS (symbols) to represent data and instructions. Such program eliminates problems associated with machine language. Computer cannot execute directly a program written in assembly language; it requires a translator called assembler. Assembler is a special program designed to translate a program written in assembly language to a machine language equivalent.

Advantages of Assembly Language

It allows complex jobs to run in a simpler way.

It is memory efficient, as it requires less memory.

It is faster in speed, as its execution time is less.

It is mainly hardware oriented.

It requires less instruction to get the result.

It is used for critical jobs.

Disadvantages of Assembly Language

It is machine dependent; the programmer must be knowledgeable in both subject area and the operations of the machine.

It is cumbersome though less cumbersome than that of machine language.

Very expensive to develop

It consumes time

High Level Language

A high-level language is a problem-orientated programming language. The source programs are written in human readable languages like English instead of mere symbols. In other words, a high-level language is a convenient and simple means of describing the information structures and sequences of actions required to perform a particular task.

Advantages of High-Level Language

No knowledge of the computer in which the program will be run is required

The programs are portable

Very easy to learn and write

Disadvantages of High-Level Language

It takes additional translation times to translate the source to machine code.

High level programs are comparatively slower than low level programs.

Compared to low level programs, they are generally less memory efficient.

Cannot communicate directly with the hardware

Features of High-Level Language

Machine independent

Problem oriented

Ability to clearly reflect the structure of program written in it.

Readability

Programs are portable.

Examples of High-level Languages are FORTRAN, COBOL, QBASIC, VISUAL BASIC, JAVA, PASCAL etc.

Characteristics of a Good Program

Portability/Transferability: Must be able to work on any computer machine.

Reliability: It can be relied upon to do what it is expected to do.

Efficiency/cost saving: It must not cost more than its benefits and enables problem to be solved appropriately, quickly and efficiently.

Clarity and Simplicity: It should be as simple as possible to understand.

Understandability/Readability: It must be readable and understandable by other programmers and end users.

Flexibility/Adaptability/Maintainability: A good program must be flexible adaptable and maintainable in order to suit user's need.

Modification must be possible and very easy.

Phases of Program Development (Programming)

The process of producing a computer program (software) may be divided into eight phases or stages:

- 1) Problem definition/Analysis
- 2) Selection or development of an algorithm
- 3) Designing the program
- 4) Coding the programming statements

- 5) Compiling/Compilation stage
- 6) Testing/Running and Debugging the program
- 7) Documentation.
- 8) Maintenance

1) Problem Definition/Analysis Stage: There is need to understand the problem that requires a solution. The need to determine the data to be processed, form or type of the data, volume of the data, what to be done to the data to produce the expected/required output.

2) Selection or development of an algorithm: An algorithm is the set of steps required to solve a problem written down in English language.

3) Designing the program: In order to minimize the amount of time to be spent in developing the software, the programmer makes use of flowchart. Flowchart is the pictorial representation of the algorithm developed in step 2 above. Pseudocode IPO chart (input processing output) and HIPO (Hierarchical Input Processing and Output) chart may be used in place of flowchart or to supplement flowchart.

4) Coding the statement: This involves writing the program statements. The programmer uses the program flow chart as a guide for coding the steps the computer will follow.

5) Compiling: There is need to translate the program from the source code to the machine or object code if it is not written in machine language. A computer program is fed into the computer first, then as the source program is entered, a translated equivalent (object program) is created and stored in the memory.

6) Running, Testing and Debugging: When the computer is activated to run a program, it may find it difficult to run it because errors (syntax, semantics or logic, or runtime) might have been committed. Manuals are used to debug the errors. A program that is error free is tested using some test data. If the program works as intended, real life data are then loaded.

7) Documentation: This is the last stage in software development. This involves keeping written records that describe the program, explain its purposes, define the amount, types and sources of input data required to run it. List the Departments and people who use its output and trace the logic the program follows.

8) Maintenance: All the activities that occur after the completion of the program come under the program maintenance. Program maintenance includes the following: Finding and correcting the errors; Modifying the program to enhance it – i.e., adapting to some new concepts or when there is a change in the hardware or operating system; Update the documentation; Add new features and functions; Remove useless or redundant parts of code.

Structured programming Concept

In structured programming design, programs are broken into different functions these functions are also known as modules, subprogram, subroutines or procedures.

Each function is design to do a specific task with its own data and logic. Information

can be passed from one function to another function through parameters. A function can have local data that cannot be accessed outside the function's scope. The result of this process is that all the other different functions are synthesized in another function. This function is known as main function. Many of the high-level languages support structured programming.

Structured programming minimizes the chances of the function affecting another. It allows for clearer programs code. It made global variables to disappear and replaced by the local variables. Due to this change one can save the memory allocation space occupied by the global variable.

Its organization helps in the easy understanding of programming logic. So that one can easily understand the logic behind the programs. It also

Helps the newcomers of any industrial technology company to understand the programs created by their senior workers of the industry.

The languages that support Structured programming approach are:

- C
- C++
- Java
- C#
- Pascal

Python

Fortran

Cobol

Advantages of Structured programming

It is user friendly and easy to understand.

Similar to English vocabulary of words and symbols.

It is easier to learn.

They require less time to write.

They are easier to maintain.

These are mainly problem oriented rather than machine based.

Program written in a higher-level language can be translated into many machine languages and therefore can run on any computer for which there exists an appropriate translator.

It is independent of machine on which it is used, i.e., programs developed in high level languages can be run on any computer.

Disadvantages of Structured Programming

Structured programming codes implemented in high-level language has to be translated into the machine language by translator and thus a price in computer time is paid.

The object code generated by a translator might be inefficient compared to an equivalent assembly language program.

Data type are proceeds in many functions in a structured program. When changes

occur in those data types, the corresponding change must be made to every location that acts on those data types within the program. This is really a very time consuming task if the program is very large.

In a structured program, each programmer is assigned to build a specific set of functions and data types. Since different programmers handle separate functions that have mutually shared data type, other programmers in the team must reflect the changes in data types done by the programmer in data type handled. Otherwise, it requires rewriting several functions.

Programming paradigms

Solving a programming problem requires choosing the right concepts. All but the smallest toy problems require different sets of concepts for different parts of the program. A programming paradigm, or programming model, is an approach to programming a computer based on a mathematical theory or a coherent set of principles. It is a way of conceptualizing what it means to perform computation and how tasks to be carried out on the computer should be structured and organized.

Programming languages are used to realize programming paradigms.

Examples of programming paradigms are: imperative, functional, logical, object-oriented. Most popular languages are imperative and use structured programming techniques. Structured programming techniques involve giving the code you write structures, these often involve writing code in blocks such as sequence (code executed line by line), selection (branching statements such as if..then..else, or case) and repetition (iterative statements such as for, while, repeat, loop).

Imperative paradigm

This paradigm is based on the ideas of a Von Neumann architecture. A command has a measurable effect on the program and the order of commands is important. First do this and next do that. Its main characteristics are incremental change of the program state (variables) as a function of time; execution of commands in an order governed by

control structures; and the use of procedures, abstractions of one or more actions, which can be called as a single command. Examples: Fortran, Algol, Basic, C, Pascal.

Functional paradigm

This paradigm is based on mathematics and theory of functions. The values produced are non-mutable and plays a minor role compared to imperative program. All computations are done by applying functions with no side effects. Functions are first class citizens. Evaluate an expression and use the resulting value for something.

Example: Haskell.

Logical paradigm

The logic paradigm fits well when applied in problem domains that deal with the extraction of knowledge from basic facts and relations. Is based on axioms, inference rules, and queries. Program execution becomes a systematic search in a set of facts, making use of a set of inference rules. Examples: Prolog and List.

Object-oriented paradigm

Data as well as operations are encapsulated in objects. Information hiding is used to protect internal properties of an object. Objects interact by means of message passing. In most object-oriented languages, objects are grouped in classes and classes are organized as hierarchies. Examples: C++, Java.

Other Paradigms

Other paradigms include Visual paradigm, Constraint based paradigm, Aspect oriented paradigm and Event-oriented paradigm.

Paper 4 of 4 — CSC 311 MATERIAL

Level 300 • First Semester • Handout

Lecture notes on

Structured programming

Csc 311

FIRST SEMESTER (2023/2024 SESSION)

DEPARTMENT OF COMPUTER SCIENCE,

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES

COURSE OUTLINE:-

□ Structured Programming Elements.

□ Structured Design Principles.

□ Abstraction and Modularity.

□ Stepwise Refinement.

□ Structured Design Techniques.

Lecture one

Structured

Programming

Elements

INTRODUCTION

Solving a programming problem requires choosing the right concepts. A programming paradigm (or programming model) is an approach to programming a computer based on a mathematical theory or a coherent set of principles. It is a method of conceptualizing what it means to perform computation and how tasks to be carried out on the computer should be structured and organized.

Each paradigm consists of certain characteristics, features, and opinions about how common programming problems should be tackled. Programming paradigms are not languages or tools. You can't "build" anything with a paradigm. They're more like a set of ideals and guidelines that many people have agreed on, followed, and expanded upon. Programming languages are used to realize programming paradigms. Examples of programming paradigms are: imperative, functional, logical, object-oriented.

While programming refers to the process of writing programs that convert problem solutions into set of instructions to enable a computer to do a certain task, structured programming refers to the process of writing programs that are easier to test, debug, modify and maintain by enforcing a modular approach, which breaks a large complex problem into sub-problems.

Most popular languages are imperative and use structured programming techniques. Structured programming techniques involve giving the code you write structures, these often involve writing code in blocks such as sequence (code executed line by line), selection (branching statements such as if-then-else, or case) and repetition (iterative statements such as for, while, repeat, loop).

Definition: Structured programming (sometimes known as modular programming) is a programming paradigm that facilitates the creation of programs with readable code and reusable components. All modern programming languages support structured programming, but the mechanisms of support, like the syntax of the programming languages, varies.

UNSTRUCTURED VS. STRUCTURED PROGRAMMING

In unstructured programs, the statements are executed in sequence (one after the other) as written.

This type of programming uses the GoTo statement which allows control to be passed to any other section in the program. When a GoTo statement is executed, the sequence continues from the target of the GoTo. Thus, to understand how a program works, you have to execute it. This often makes it difficult to understand the logic of such a program.

Structured programming approach was developed as a solution to the challenges posed by unstructured programming. Structured programming frequently employs a top-down design model, in which developers break the overall program structure into separate subsections.

A defined function or set of similar functions is coded in a separate module or sub-module. This means that code can be loaded into memory more efficiently and that modules can be reused in other programs. After a module has been tested individually, it is then integrated with other modules into the overall program structure. Program flow follows a simple hierarchical model that employs control constructs (such as "for," and "while.", etc). Use of the "GoTo" statement is discouraged.

Consider the below example (Unstructured vs Structured):

Unstructured (BASIC) Structured (C)

```
10 I=0 for(i=0; i < 10; i=i+1) {  
20 PRINT "Hello, World!" printf("Hello, world!");  
30 I=I+1 }  
40 IF I<10 THEN GOTO 20
```

In the BASIC example the flow of the code is controlled using the GOTO statements and there is no apparent "structure" to the code, whereas in the C example the print statement is a child of the for statement, giving the program a more tree like structure.

WHY USE STRUCTURED PROGRAMMING?

Whether you are a novice or skilled programmer, you will definitely face difficulties in understanding unstructured or spaghetti code. Consider an unstructured program with over 1000 instructions. If you encounter an error while running it, it becomes challenging to locate the error due to the program's unorganized structure.

Now, consider you encounter an error in a structured program with organized instructions. You will find it easy to locate the error and save significant time. This is where the role of structured programming comes into play.

□ In structured programming, programs are broken into different functions also known as modules, subprogram, subroutines or procedures. Each function is design to do a specific task with its own data and logic. Information can be passed from one function to another

function through parameters. A function can have local data that cannot be accessed outside the function's scope. This function is known as main function.

- Structured programming encourages dividing an application program into a hierarchy of modules or autonomous elements. This means that control is passed downwards only through the hierarchy.
- Structured programming linearizes the flow of control through a program. This means all the operations and execution paths in the program follow a well-defined order.
- Structured programming replaced global variables with local variables. Due to this change one can save memory allocation space occupied by the global variable.

ELEMENTARY STRUCTURES OF STRUCTURED PROGRAMMING

- Block – A block consists of a sequence of statements that is treated as a single unit in its enclosing block. These blocks are typically defined by curly braces { } and can contain one or more statements or expressions, which are executed in the order they appear.
- Nesting – Nesting involves creating a structure inside another structure, called a nested structure. A nested structure is one that contains one or more members that are themselves structures. Nested structures are useful for organizing complex data and can help improve a program's readability and maintainability.
- Subroutines – Subroutines are the functions, methods, procedures, or subprograms containing a set of sequential instructions to perform a specific task. It is possible to use a subroutine repeatedly in a program. Each subroutine can work independently. For instance, consider you write a subroutine to add two numbers. You can use this subroutine anywhere in a program where there is a need to add two numbers.
- Sequence – A sequence is an ordered arrangement of instructions, subroutines, or statements. It has a single entry (the first line of a program) and exit (the last line of a program) point. The sequence is executed in a linear fashion. It starts executing the statements from top to bottom, similar to how they are written.
- Selection – Selection refers to executing blocks of code based on the outcome of a condition. It involves the use of the keywords called conditional statements, such as if, then, else, and endif. If the specified condition with the “if” statement holds true, the control flow moves to statements present in that block.
- Iteration – An iteration repeatedly executes a block of code a certain number of times or as long as the condition is met. We use ‘for,’ ‘while,’ and ‘do-while’ loops to iterate a specific block of code. With the ‘for’ loop, it is possible to specify the number of times you want to execute a code block. It requires you to specify the condition in the beginning. The same goes for the while loop. However, the do-while loop evaluates the condition at the end of the code block.
- Early Exit: This refers to a coding strategy where a function, method, or loop terminates its execution as soon as it satisfies its intended condition, rather than waiting for the entire function to complete. Early exit can lead to a more efficient execution of code and can also improve the readability and clarity of code. Examples include:

- break - Exits loop or switch immediately.
- continue - Moves to the next iteration of a loop.
- return - Exits subroutine immediately.
- exit - Exits program immediately.

TYPES OF STRUCTURED PROGRAMMING

There are three categories of structured programming:

1. Procedural programming: Procedural programming encourages the use of procedures (subroutines or functions) to perform computations. A procedure is any function, routine, or subroutine containing a sequence of instructions or sequential steps to be performed.

It is possible to call any procedure in a program at any given point during the program execution. Procedural programming follows the top-down approach.

A program in procedural programming consists of two separate entities – data and functions/procedures working on that data.

2. Object-oriented programming (OOP). Object-oriented programming is a widely used paradigm that organizes a program or software into objects rather than functions and logic.

An object contains data and code. The data is present in the form of attributes or properties, while the code is in the form of procedures (methods or functions).

The primary goal of object-oriented programming is to combine data and procedures working on that data. No other functions or part of the code can access that data.

3. Model-based programming. Database programming languages, like SQL, are the best examples of model-based programming. Every unit of code in a database programming language is associated with the steps to access or update a database.

FEATURES OF STRUCTURED PROGRAMMING

In structured programming, readable code is created using control

Structured Control

flow constructs like loops (while, for) and selection (if, else).

It encourages reusable elements like subroutines and routines for

Reusable Components

efficient code organization.

Aims to simplify complex software by emphasizing clear and

Simplifies Complexity

logical sequences.

Replaces go-to statements with loops and subroutines for improved

Avoids Jumps

clarity and efficiency.

Ensures programs follow a well-defined, linear order for smoother

Linear Execution

execution.

Top-Down Approach Adheres to a top-down approach, outlining clear execution paths.

Wide Language Modern languages like C, C++, Java, and Python embrace Support structured programming, adapting syntaxes accordingly.

Lecture TWO

STRUCTURED

DESIGN

PRINCIPLES

The structured programming concept was formalized in 1966 by Corrado Böhm and Giuseppe Jacopini, who demonstrated theoretical computer program design through loops, sequences and decisions. In the late 1960s-early 1970s, Edsger W.Dijkstra developed structural programming functionality as a widely used method, in which a program is divided into multiple sections with multiple exits and one access point.

In the process of coding, the lines of code keep multiplying, thus, the size of the software increases. Gradually, it becomes next to impossible to remember the flow of the program. If one forgets how software and its underlying programs, files, and procedures are constructed, it then becomes very difficult to share, debug, and modify the program. The solution to this is the structured programming design.

Structured programming design involves three main concepts:

1. Top-Down Analysis

Programs are written to provide a solution to any type of problem. However, developing a programmatic solution for larger problems can be complicated. We might think, where do we start from, how do we proceed, and where do we terminate.

□ Top-down analysis is a method of problem-solving whose approach is to subdivide the program into smaller tasks for easing the analysis. This includes:-

- i. Subdivision of program – The entire problem is broken down into smaller parts so that it can be analyzed and an appropriate solution can be devised for it.
- ii. Hierarchy of task – The subdivided parts of the major problem are further broken into task thus creating a hierarchy.

Steps For Top-Down Analysis

Step One □ Define the complete scope of problem to be solved. It includes determining the

□ Input: What data is to be processed?

□ Process: What to do on the input data and what type of processing is needed to be done?

□ Output: What information should the program produce?

Step Two □ Divide the problem into two separate parts.

Step Three □ Define the scope of each subdivided parts and divide them further into tasks.

Step Four □ Repeat step three till a simple task is reached, which cannot be further subdivided.

2. Modular Programming

□ This is a programming design style where the program is broken into smaller independent units called modules, and each module is developed independent of each other.

- Each module accomplishes one function and contains all the source code and variables needed to accomplish that function.
- Modular programming discourages jumps using 'goto' statements in the program, which often makes the program flow non-traceable. Jumps are prohibited and modular format is encouraged in structured programming.
- Every module has an entry point to which the control is transferred to, and only one exit point from which the control is returned back to the calling function.

3. Structured Coding

- Structured coding sub-divides the modules into further smaller units of code in the order of their execution. While structured programming uses control structure to control the flow of the program, structured coding uses control structure to organize its instructions in definable patterns.
- Structured coding helps in the easy understanding of programming logic. Usually, the flow of execution of programs that do not require logic is normal and straight forward. However, if logic is required, then there is a need to alter the normal flow of execution. This alteration is done with the use of control transfer statements (like “if” and “switch-case”) and loops (like “for”, “while”, “do-while”, etc) so that one can easily understand the logic behind the programs.

ADVANTAGES OF STRUCTURED PROGRAMMING

1. Programs are user friendly and easier to read, learn, and understand.
2. Programs are less likely to contain logic errors.
3. Programs are more easily maintained.
4. It requires less time to write programs.
5. Locating and debugging errors is done easily.
6. Higher productivity during program development.
7. It is similar to English vocabulary of words and symbols.
8. It is mainly problem oriented rather than machine based.
9. It is machine independent and can run on any computer.
10. Most high-level programming languages support structured programming.
11. Programs developed in high-level languages can be translated into many machine languages, and can run on any computer with an appropriate translator.

DISADVANTAGES OF STRUCTURED PROGRAMMING

1. Programs developed in high-level languages has to be translated into the machine language by a translator, and thus a price in computer time is paid.
2. The object code generated by a translator might be inefficient compared to an equivalent assembly language program.
3. Programs depend on changeable factors, like data types. When changes occur in those data types, the corresponding change must be made to every location that acts on those data types within the program. This is really time consuming.

4. Developing applications using this approach takes time, as it is language-dependent. This is not the case with assembly languages, as they are pre-programmed for machines.
5. It results in greater memory usage due to the use of calls to modules and subroutines.

Lecture four

MODULARITY

INTRODUCTION

Modular programming is a general programming concept where developers separate program functions into independent pieces. These pieces then act like building blocks, with each block containing all the necessary parts to execute one aspect of functionality. When the blocks, or modules, are put together, they make up the executable program.

Definition: Modular programming is a development process of subdividing a computer program into separate, independent and interchangeable sub-programs (modules). Similar functions are grouped in the same module which improves the testability, maintainability and reusability by other applications.

While most people talk about modularity at a file/folder/repository level, modularity works on multiple levels which includes:

- Functions within files
- Files within repositories/libraries
- Libraries/repositories within projects

Modularity is about making building blocks, and these building blocks are also made up of other smaller building blocks. Each block is separate, strong, and testable, so you can stack them together at the end to create an application.

MODULARITY DESIGN PRINCIPLE

The basic principle of Modularity is that “Systems should be built from cohesive, loosely coupled components (modules)” which means a system should be made up of different components that are united and work together in an efficient way, and such components have a well-defined function.

A module is a part of a modular system. To define a modular system, several properties or criteria are there under which we can evaluate a design method of a module while considering its abilities. Some of them are given below:

1. Modular Decomposability –

Decomposability simply means to break down something into smaller pieces. Modular decomposability means to break down the problem into different sub-problems in a systematic manner. Solving a large problem is difficult sometimes, so the decomposition helps in reducing the complexity of the problem, and sub-problems created can be solved independently.

2. Modular Composability –

Composability simply means the ability to combine modules that are created. It's the principle of system design that deals with the way in which two or more components are related or connected to each other. Modular composability means to assemble the modules into a new system (by

means connecting the combined components).

3. Modular Understandability –

Understandability simply means the capability of being understood, quality of comprehensible. Modular understandability means to make it easier for the user to understand each module so that it is very easy to develop software and change it as per requirement. Sometimes it's not easy to understand the process models because of its complexity and its large size in structure. Using modularity understandability, it becomes easier to understand the problem in an efficient way without any issue.

4. Modular Continuity –

Continuity simply means unbroken or consistent or uninterrupted connection for a long period of time without any change or being stopped. Modular continuity means making changes to the system requirements that will cause changes in the modules individually without causing any effect or changes in the overall system or software.

5. Modular Protection –

Protection simply means to keep something safe from any harms, to protect against any unpleasant means or damage. Modular protection means to keep safe the other modules from the abnormal condition occurring in a particular module at run time. The abnormal condition can be an error or failure also known as run-time errors. The side effects of these errors are constrained within the module.

MODULE TYPES

Modules are important because they allow us to take large complicated programs and to divide them into smaller manageable pieces. Because the module is a smaller piece of the overall program, we can concentrate on what we want it to do and test it to make sure it works properly. Generally, modules fall into two categories:

□ Program Control Modules – are designed to simply sub-divide and control the program. These modules are unique to the program being written. Other programs may use similar modules, maybe even modules with the same name, but the content of the modules is almost always very different.

□ Specific Task Modules – are designed to be used with several programs. These modules perform a specific task and thus are usable in many different programs because the other programs also need to do the specific task. Specific task modules are sometimes referred to as building blocks. Because they are already coded and tested, we can use them with confidence to more efficiently write a large program.

SOFTWARE MODULARITY

Software modularity is measured by how well a software is decomposed into smaller pieces with standardized interfaces. The intent of software modularity is different depending on the strategic targets of the software owner. Here are some examples of strategy and intent:

□ When software and hardware are combined, software portability might be a key target of modularity. Software portability is defined as software that is independent of the hardware it runs on. This autonomy can be achieved with carefully designed interfaces that abstract

the hardware level from the application level.

□ Automated testing is typically a major concern for software that is changed often. By being able to test faster and spot issues earlier, less time and resources can be spent on integrating the changes. Automated testing is something that can be achieved better with smart software modularity. By isolating changing parts with interfaces, we can test smaller chunks which is faster both in testing and in finding bugs.

□ For software where updates and features are implemented on an installed base, Over-the-Air (OTA) update capability is the tale of the century. To push software updates in a controlled and efficient matter, it is essential to separate frequently updated modules from fundamental critical modules where changes are high risk. Updating small chunks of the software is also faster and less resource consuming.

SOFTWARE MODULE

In software, the term Module has a wide range of different definitions, much like the term Component. Even though it is a term with many different definitions, there are similarities in the descriptions. Let's have a look at some of these definitions.

A Software Module is Independent

A software module is independent of other modules in the sense that it is possible to develop and maintain a module with minimal to no impact on other modules. It does not mean that a module has no dependencies on other modules when running a program.

This autonomy is an important aspect of a software module because it helps to increase productivity. Independent modules enable software engineering teams to be independent of each other when creating, maintaining, and testing the code.

A Software Module is Reusable

A module can be reused in different systems, and it is also interchangeable (replaceable) within a system. Reusing modules in different systems or products without modification is very beneficial and will help shorten the time it takes for the software to hit the market.

A Software Module is Interchangeable

Interchangeable means you can replace a module without impacting the rest of the system. This flexibility is essential when upgrading a system because of new features or bug corrections. If you can limit changes to one or a few modules, less time is needed to test and implement changes. More frequent and smaller releases, in terms of impacted parts of the system, enable agility. Smaller releases also mean that you take less risk when releasing new features.

HOW CAN A SOFTWARE MODULE BE INDEPENDENT, REUSABLE, AND INTERCHANGEABLE?

A module encapsulates functionality and data.

Encapsulation is about hiding the implementation details of a module. The less a module knows about other modules, the less dependent it will be on changes in other modules. Other modules can only access functionality and data via the provided interface. Generally, it is good to encapsulate as much as possible.

A module provides an interface to other modules for accessing its functionality and data.

The software module interface should be as abstracted and as small as possible. Fewer details mean looser coupling between software modules and make them more independent, reusable, and interchangeable.

A module requires interfaces provided by other software modules.

To realize the functionality of a software module, it must sometimes use the provided interfaces of other modules. A software module that uses another module through its interface has a dependency on that other module.

Let's illustrate.

Module B1 provides Interface B to Module A and requires Interfaces C and D from Module C and D respectively.

Assume we have a new Module B2 with improved performance. If Module B2 provides the same interface and requires the same interfaces as B1, then it is interchangeable with B1.

A module that defines provided and required interfaces become reusable in other systems, e.g., Module B1, can be reused in another system as long as the required interfaces are provided.

MODULARITY IN JAVA

Modularity helps Java developers to perform functionality testing on the go while the development process is ongoing simultaneously. With modularity, the partitioning of the software environment in different modules makes the entire process more optimized and reduces the coupling effect.

Modularity helps a Java developer in three ways,

- **Reusability:** Reusability can help developers save time using the same code while creating a newer version of the software or app. With modularity in Java, you can reuse the modules elsewhere than the developed software initially.
- **Stability:** Modularity helps sustain the stability of software while making changes in individual sections of the code. It means that the new modifications remain hidden from consumers and can be easily tested without affecting the existing software.
- **Parallel development:** Java developers can quickly develop modules parallel to each other. Once all the modules are developed, they can be combined through an interface to create a suite. Using parallel development can save time.

ABSTRACTION AND MODULARITY

Abstraction and modularization are both important concepts in computer science and software engineering.

Abstraction involves simplifying complex systems by representing the essential features without including the background details. It allows developers to focus on the necessary aspects of a system while hiding unnecessary details.

On the other hand, modularization involves breaking down a system into smaller, manageable and independent modules or components. Each module performs a specific function and can be

developed, tested, and maintained independently.

These two techniques go hand-in-hand; if the interface between modules does not function well (modularity without abstraction), the effect of having modularity at all becomes less convenient to the programmer than the lack of modularity. Both concepts are essential for building complex and maintainable software systems.

Lecture Three

ABSTRACTION

INTRODUCTION

Abstraction is the process of hiding the internal details of an application from the outer world. Abstraction is used to describe things in simple terms. It's used to create a boundary between the application and the client programs. Abstraction defines a model to create an application component. The implementation of abstraction depends on the language-specific features and processes.

Real Life Examples – Abstraction is present in almost all the real-life machines.

□ Your car is a great example of abstraction. You can start a car by turning the key or pressing the start button. You don't need to know how the engine is getting started, what all components your car has. The car internal implementation and complex logic is completely hidden from the user.

□ We can heat our food in Microwave. We press some buttons to set the timer and type of food. Finally, we get a hot and delicious meal. The microwave internal details are hidden from us. We have been given access to the functionality in a very simple manner.

□ Another example is the washing machine. It has a variety of buttons and knobs that allows you to wash different types of clothes. Depending on your laundry needs, you can choose from a range of washing programs available on the machine's interface. It is not necessary to have knowledge about the internal mechanism of a washing machine, such as the functioning of drums, water valves, thermostat and springs in order to effectively wash your clothes. Fortunately, the washing machine's user interface is designed to conceal and simplify all the intricate workings of the actual mechanism.

TYPES OF ABSTRACTION

There are primarily two types of abstraction implemented in OOPs.

1. Data Abstraction – Data abstraction is the simplest form of abstraction. When working with OOPS, you primarily work on manipulating and dealing with complex objects. This object represents some data but the underlying characteristics or structure of that data is actually hidden from you.

Let's go back to our washing machine example.

Let's say my current laundry needs is to remove some stubborn shawarma stain from my white T-shirt. But, using normal liquid soap did not give me the preferred result. Luckily, I have a liquid bleach I can add to help remove the stubborn stain. So, I can directly add the liquid bleach and the washing machine treats it as just any other regular liquid soap. In this case, the liquid bleach

itself is an abstraction of the original data (the liquid soap). I can use the liquid bleach directly without worrying about how the liquid soap was made to interact with it.

Therefore, data abstraction refers to hiding the original data entity via a data structure that can internally work through the hidden data entities.

2. Process Abstraction – Where data abstraction works with data, process abstraction does the same job but with processes. In process abstraction, the underlying implementation details of a process are hidden. We work with abstracted processes that under the hood use hidden processes to execute an action.

Circling back to our washing machine example.

Our washing machine has a function to thoroughly clean stains without damaging the fabric. We activate this function by pressing a button or turning a knob on the washing machine interface, which sends it a command to begin the wash cycle. During the wash cycle, the tub moves back and forth, agitating the clothes and supplying the mechanical action that scrubs out stains. Dirty water drains out of the drum, followed by a short spin cycle to get out even more water. The washer refills with clean water for the rinse cycle, which removes any remaining detergent and dirt. The washer drains again, followed by a spin cycle (or dryer) that gets as much water out of the clothes as possible.

A single process of starting the wash cycle was known to us, but internally it implements multiple other processes that were actually abstracted from us. This is process abstraction in a nutshell.

MULTIPLE LAYERS OF ABSTRACTION

Today's applications are often programmed in multiple layers of abstraction. For example, a program can be written by selecting menu options and dragging icons around the screen. These movements may be translated into a high-level language that is translated into a low-level language, which is translated into machine language. Abstraction handles the amount of complexity by which a system is viewed or programmed. The higher the level of abstraction, the less detail. The lower the level of abstraction, the more detail.

In a typical structured program, there are many important levels of abstraction.

□ At the highest level a program is viewed as a “community” of objects that must interact with each other in order to achieve their common goal. This notion of community finds expression in two distinct forms.

First there is the community of programmers, who must interact with each other in the real world in order to produce their application. Secondly, there is the community of objects that they create, which must interact with each other in a virtual universe in order to further their common goals. Each object in this community provides a service that is used by other members of the organization.

At this highest level of abstraction, the important features to emphasize are the lines of communication and cooperation, and the way in which the members must interact with each other.

□ The next level of abstraction is not found in all structured programs, nor is it supported in

all programming languages. However, many programming languages permit a group of objects working together to be combined into a unit. Examples of this idea include packages in Java, name spaces in C++, units in Delphi, etc. The unit allows certain names to be exposed to the world outside the unit, while other features remain hidden inside the unit.

□ The next two levels of abstraction deal with the interactions between two individual objects (named Object_A and Object_B). This interaction is similar to the communication between a client and a server.

- o Object_B (Server): At this level of abstraction, a good structured program design should be able to describe and discuss the services that Object_B provides without reference to any actions that Object_A may perform in using those services. One can think of this as being like a billboard advertisement. Often, this level of abstraction is represented by an interface, a class-like mechanism that defines behavior without describing an implementation.

- o Object_A (Client): The next level of abstraction considers a concrete implementation of the abstract behaviour. That is, it deals with the way in which the services provided by Object_B are being realized.

The lowest level of abstraction considers a single task in isolation; that is, a single method. Concerns at this level of abstraction deals with the precise sequence of operations used to perform just this one activity. For example, we might investigate the technique used to sort an array of integer numbers in descending order.

Note: The history of programming has experienced rising levels of granularity. Decades ago, programmers manipulated individual bits of codes. Then the assembly language was invented, and programmers began to write instructions that were equivalent to a few bytes.

The advantage was clear. Instead of thinking in terms of essentially meaningless 1s and 0s, you could think in terms of what the computer was doing on a functional level (e.g., move this value to that memory location, multiply these two bytes together, etc). This is called raising the level of abstraction.

Every time you raise the level of abstraction in a programming language, you get a clearer and more structured program. The language at which you communicate with the computer can also be altered into something closer to the way we communicate in English.

ABSTRACTION IN OOP LANGUAGES (JAVA)

Objects in an OOP language provide an abstraction that hides the internal implementation details. Similar to the microwave in your kitchen, you just need to know which methods of the object are available to call and which input parameters are needed to trigger a specific operation. But you don't need to understand how this method is implemented and which kinds of actions it has to perform to create the expected result.

Abstraction, in Java programming language, refers to using simplified classes, rather than complex implementation code, to access objects. This is done by hiding the implementation details of a class (data members) and presenting a clean, easy-to-use user interface through the

class member functions.

ABSTRACTION CAN BE ACHIEVED WITH ABSTRACT CLASSES

An abstract class is a class that is declared with an abstract keyword. The abstract keyword is a non-access modifier.

An abstract method is a method that is declared without implementation. Abstract methods can only be used in an abstract class, and it does not have a body.

Example:

```
// Abstract Class
abstract class Animal {
    public abstract void makeSound();

    // Abstract Class To access the abstract class, it must be
    abstract class Animal { inherited from another class (subclass or
    public abstract void makeSound();
    concrete class) with the extends keyword.
    // Subclass (inherit from Animal)
```

The body of the abstract method is provided

class Dog extends Animal { by the subclass (inherited from).

```
public void makeSound() {
    System.out.println("Jack is Barking");
```

Below is the completion of our example.

```
// Abstract Class
abstract class Animal {
    Note:
    public abstract void makeSound();
} 1. Abstraction lets you focus on what the
// Subclass (inherit from Animal) object does instead of how it does it.
class Dog extends Animal { 2. There can be no object of an abstract class
public void makeSound() {
    (an abstract class cannot be directly
    System.out.println("Jack is Barking");
} instantiated with the new operator).
} 3. Any class that contains one or more
// Main Class
```

abstract methods must also be declared

```
public class AbstractionExample {
    public static void main(String[] args) {
        with an abstract keyword.
```

Animal myDog = new Dog(); 4. An abstract class can have both abstract
myDog.makeSound(); and regular methods.

ABSTRACTION CAN BE ACHIEVED WITH INTERFACES

Interfaces are another method of implementing abstraction in Java. The key difference is that, by using interfaces, we can achieve 100% abstraction in Java classes. An interface is a completely "abstract class" that is used to group related methods with empty bodies. To access the interface methods, the interface must be "implemented" by another class with the implements keyword (instead of extends). The body of the interface method is implemented in the subclass.

```
interface Animal {  
    public void makeSound();  
    public void sleep();  
    class Lion implements Animal {  
        public void makeSound() {  
            System.out.println("Simba is Roaring");  
        }  
        public void sleep() {  
            System.out.println("Simba is Sleeping");  
        }  
    }  
    public class AbstractionExample {  
        public static void main(String[] args) {  
            Animal myLion = new Lion();  
            myLion.makeSound();  
            myLion.sleep();  
        }  
    }  
}
```

ALGORITHM TO IMPLEMENT ABSTRACTION IN JAVA

1. Determine the classes or interfaces that will be part of the abstraction.
2. Create an abstract class or interface that defines the common behaviours and properties of these classes.
3. Define abstract methods within the abstract class or interface that do not have any implementation details.
4. Implement subclasses that extend the abstract class or implement the interface.
5. Override the abstract methods in the subclasses to provide their specific implementations.
6. Use the subclasses to implement the program logic.

ADVANTAGES OF ABSTRACTION

1. Avoids code duplication and increases code reusability.
2. Improves the maintainability of the application.
3. Hides implementation details and exposes only relevant information.
4. Reduces complexity by providing a clear and simple interface to users.
5. Increases security by preventing access to internal class details.
6. Supports modularity, as complex systems can be divided into smaller and more manageable parts.
7. Allows for flexibility in the implementation of a program, as changes to the underlying implementation details can be made without affecting the user interface.

DISADVANTAGES OF ABSTRACTION IN JAVA

1. It may limit the flexibility of the implementation, making it difficult to understand how the system works.

2. It can add unnecessary complexity to the code if not used appropriately, leading to increased development time and effort.
3. Unfamiliarity with abstraction layers and implementation details may result in writing codes that are difficult to understand and hard to debug.
4. Abstraction overuse can result in decreased performance due to the additional layers of code and indirection.